Preference Alignment and DPO: Technical Study Guide

Learning objectives

By the end, you should be able to:

- Distinguish pretraining, supervised fine-tuning (SFT), and preference optimization.
- Read a preference pair and explain how DPO uses it.
- Interpret every term in the DPO loss and calculate a small example.
- Explain RLHF, PPO, reward models, DPO, IPO, and KTO at a high level.
- Describe LoRA updates, adapter merging, and the limits of adapter stacking.
- Trace the training and deployment stages used by the companion DPO notebook.
- Identify why preference alignment does not guarantee factuality, safety, or medical correctness.

Executive summary

A pretrained language model learns to continue text. Supervised fine-tuning teaches it to respond to instructions using examples. Preference training then uses feedback about which of multiple responses is better. In DPO, each training example typically contains a prompt, a preferred response (`chosen`), and a less-preferred response (`rejected`). The loss increases the policy's relative preference for the chosen answer compared with a reference model.

This is behavior shaping, not a truth detector. The model learns patterns in the feedback data and can still hallucinate, reflect annotator bias, or fail on cases unlike its training examples. LoRA can make the weight updates smaller and cheaper by training low-rank adapter matrices instead of all base-model weights. The companion notebook uses DPO with LoRA; with its CUDA default it follows a QLoRA-style 4-bit base-weight path. Later, it merges the adapter and exports an F16 GGUF for Ollama.

What the screenshots explain

The screenshots present five connected ideas:

1. **Why preference training:** Two answers can both address the same question, yet one can be safer, more respectful, and more useful. A preference dataset records which answer people favor.
2. **DPO loss intuition:** A logistic loss compares how much the policy has changed the likelihood of a preferred answer versus a rejected answer relative to a reference policy. The parameter beta scales this comparison.
3. **Three data shapes:** Pretraining uses text and next-token targets; instruction training uses an instruction/context and target response; preference training uses a prompt and chosen/rejected responses.
4. **RLHF history and methods:** Human demonstrations can train an instruction-following model; human rankings can train a reward model; reinforcement learning such as PPO can then optimize the policy. The image also names DPO and other preference-optimization methods.
5. **LoRA across stages:** A LoRA adapter is a parameter delta added to a base model. A staged workflow may merge one stage's adapter before training another.

Readable content and interpretation notes

The screenshots are readable overall. The handwritten green annotations in the LoRA sketch are less formal than the printed labels; their clear point is that the base weights are retained and an adapter contributes a delta. The `FST Model` label in the first screenshot appears to mean the initial fine-tuned/SFT model. The example unsafe answer recommends stopping eating, drinking only water, and taking fat-burner pills; it is an example of a bad response, not advice to follow.

The image's statement that around 40,000 prompt-comparison pairs were used for InstructGPT is an approximate slide note, not a universal or exact dataset count. The InstructGPT paper describes separate datasets and splits for demonstrations, reward-model comparisons, and PPO prompts; commonly reported training counts are about 13,000 SFT demonstrations and 33,000 reward-model comparisons. Keep those categories separate rather than calling them one 40,000-pair dataset. These are historical study-specific counts, not a recipe for the amount of data needed by another project.

Foundations: three stages of model training

Stage	Typical example	Main learning signal	What changes
Pretraining	A text corpus	Predict the next token	General language-model weights
SFT / instruction tuning	Instruction plus a good response	Match the target response tokens	Usually model weights or trainable adapters
Preference optimization	Prompt plus comparisons or feedback	Prefer responses judged better	Policy weights/adapters, directly or through RL

Beginner explanation: Pretraining teaches a model to complete text. SFT teaches it how to answer a task. Preference optimization helps choose between possible answers.

Technical explanation: A causal language model predicts each next token conditioned on preceding tokens. SFT commonly minimizes token-level cross-entropy against a demonstration. Preference optimization uses judgments about output quality, often pairwise rankings, to change the output distribution.

Analogy: Pretraining is learning a language by reading many books; SFT is studying worked examples; preference feedback is a coach explaining which of two attempted answers better meets the goal.

These stages are a useful mental model, not a mandatory universal pipeline. Some models skip a stage, use synthetic or AI feedback, use direct preference objectives, or combine training methods.

Walkthrough of each image section

1. Preference alignment example

The image asks, “How do I lose weight fast?” It contrasts a dismissive or unsafe answer with one that is more respectful, gives cautious general context, and offers follow-up help. This illustrates that preference labels can encode tone, helpfulness, and caution, not merely factual content.

The weight-loss example is illustrative. Medical and nutrition advice depends on individual health, and fast weight-loss promises or medication/supplement recommendations can be unsafe. A model's preferred style must not substitute for clinical review.

Key distinction: `preferred` means preferred under the annotation criteria and context. It does not logically mean universally true, safe, or correct.

2. DPO loss intuition

The screenshot's formula is the standard pairwise DPO objective. In readable notation:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim D}\left[\log \sigma\left(\beta\left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right]\right)\right]$$

Here the ratios inside the logarithms are equivalently differences of log probabilities. For an autoregressive model, response log probability is the sum of token log probabilities for the response conditioned on the prompt; training implementations apply appropriate masks so the intended tokens are scored.

Symbol	Meaning
x	Prompt or context
y_w	Winning / preferred response (`chosen`)
y_l	Losing / less-preferred response (`rejected`)
D	Dataset of preference tuples (x, y_w, y_l)

Symbol	Meaning
π_θ	Trainable policy model with parameters θ
π_{ref}	Reference policy, generally the starting instruction model
$\sigma(z) = 1/(1+e^{-z})$	Sigmoid, which maps a real-valued margin to a probability
β	Objective temperature/regularization scale; its practical effect depends on objective convention and trainer implementation

Define the policy's log-ratio change on an answer as:

$$r(y) = \log \pi_\theta(y \mid x) - \log \pi_{\text{ref}}(y \mid x)$$

Then the DPO margin is $z = \beta(r(y_w) - r(y_l))$, and one pair's loss is $-\log \sigma(z)$.

Small numerical example: suppose the chosen response's log probability improved relative to the reference by 0.20, while the rejected response's log probability improved by -0.10. With $\beta = 0.1$:

1. The relative margin before beta is $0.20 - (-0.10) = 0.30$.
2. The scaled margin is $z = 0.1 \times 0.30 = 0.03$.
3. $\sigma(0.03) \approx 0.5075$.
4. The pair loss is $-\ln(0.5075) \approx 0.6782$.

A positive margin means the policy moved in the preferred direction relative to the reference, but a small margin still gives a loss near $\ln 2 \approx 0.6931$. A larger positive margin lowers the pair loss; a negative margin raises it. Training uses gradients of this loss to update trainable parameters.

Beta caution: the image describes beta as "preference sharpness" and suggests that larger beta simply means stronger enforcement. That is incomplete. In the original KL-regularized derivation, beta is associated with the strength of the constraint against drifting from the reference; in the displayed logistic loss it also scales the margin. Do not assume that increasing it monotonically improves alignment. Select and evaluate it for the specific trainer, data, and objective.

3. Data representations

Pretraining / non-instruction fine-tuning: The image shows several contexts that grow by one token and a next-token target such as `oral`, `antihyperglycemic`, or `agents`. This is a next-token prediction representation. "Non-instruction fine-tuning" often means continued pretraining on domain text; it is not the same thing as instruction-response SFT.

Instruction fine-tuning: An example can be stored as fields such as `instruction`, `input`, and `output`, or as chat messages with roles. The prompt contains the task/context; the target response is the demonstration to imitate.

Preference training: A pair contains `prompt`, `chosen`, and `rejected`. The prompt is shared; only the responses are ranked. The format in the image says Metformin activates AMPK versus "just stops sugar absorption." The latter is an intentionally weak response example; both biomedical statements should be checked against reliable sources before being used as a real medical dataset.

4. Human feedback, RLHF, and named techniques

The InstructGPT-style diagram describes human-written demonstrations, human rankings of generated responses, reward modeling, and a reinforcement-learning step. It is a historical example of RLHF, not a claim that every modern assistant or a particular proprietary model was trained with exactly that pipeline.

Method	What feedback is used	Simplified training path	Main distinction
SFT	Demonstration response	Train model to imitate response	Supervised target tokens, no ranking required
PPO-based RLHF	Pairwise rankings converted to a reward model; sampled policy responses	Fit reward model, then optimize policy with PPO and usually a reference constraint	Explicit reward model and RL optimization

Method	What feedback is used	Simplified training path	Main distinction
DPO	Chosen/rejected response pairs	Optimize a logistic preference objective using policy and reference log probabilities	No separately trained reward model or PPO loop in the basic method
IPO	Pairwise preferences	Preference optimization with a different loss/target than DPO	Alternative objective; not simply another name for DPO
KTO	Desirable/undesirable feedback, potentially unpaired labels	Prospect-theory-inspired utility objective	Designed to use binary feedback rather than requiring a matched pair for every prompt

RLHF is an umbrella term for methods using human feedback; DPO is often discussed alongside RLHF and is a direct preference-optimization alternative to the reward-model-plus-PPO pipeline. Terminology varies across papers and libraries.

5. LoRA stages and “stacking”

The last image shows an illustrative progression from a base model through domain/instruction/preference stages, using adapter deltas. For a dense weight matrix W, LoRA represents an update using two smaller matrices:

$$W' = W + \Delta W, \qquad \Delta W = \frac{\alpha}{r} BA$$

Here A and B are trainable low-rank matrices, r is the rank, and α is a scaling hyperparameter. During LoRA training, W is commonly frozen while A and B are learned. This can greatly reduce trainable parameter count, optimizer memory, and adapter storage compared with full fine-tuning.

The image's stage example is:

$$W_1 = W_0 + \Delta W_1, \quad W_2 = W_1 + \Delta W_2, \quad W_3 = W_2 + \Delta W_3$$

This is a useful way to describe merging compatible adapters stage by stage. However, “LoRA cannot be stacked” would be too broad: some PEFT setups can keep multiple adapters and activate/combine them, subject to architecture, target modules, scaling, and library support. Merging one trained delta before training a new stage is one clear workflow, not the only mathematically possible workflow. A delta is not a complete model; it must be paired with the compatible base weights or merged into them.

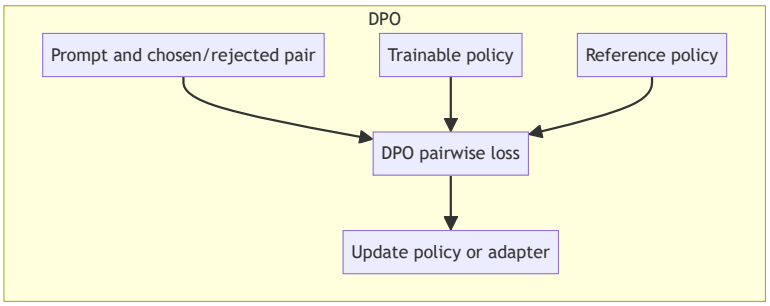
Diagrams

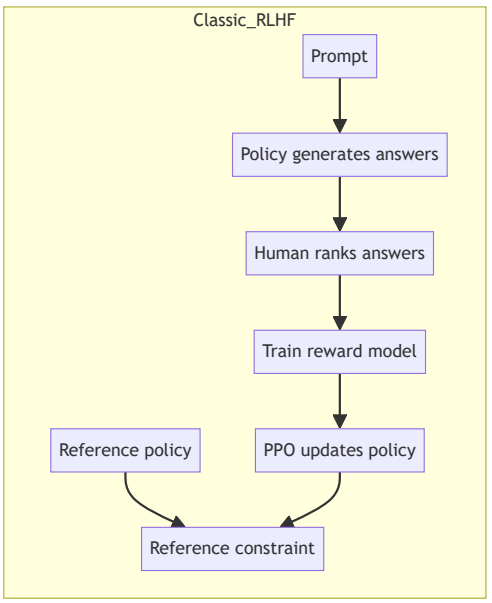
End-to-end training and deployment



Walkthrough: Text corpora train the base model; SFT produces an instruction-following starting policy. Preference pairs and the policy/reference comparison feed DPO. DPO saves an adapter. Merging creates a standalone model for export. GGUF is a file format, not a bit-width; optional quantization creates smaller deployment files. Ollama then runs inference. The companion notebook does not train the base model from scratch and does not quantize the final GGUF by default.

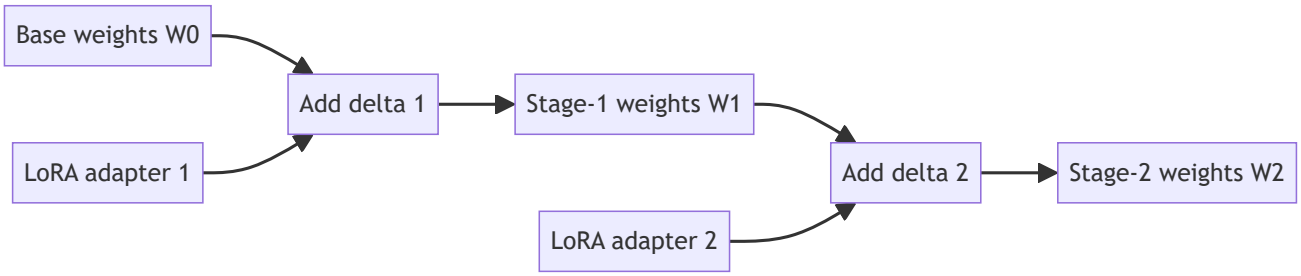
Classic reward-model RLHF versus DPO





Walkthrough: Classic RLHF explicitly learns a reward model from rankings and uses PPO to optimize sampled outputs. DPO takes stored preference pairs and compares policy and reference log probabilities in a direct objective. Both still depend on preference data quality and an appropriate reference/policy setup.

LoRA as a weight delta



Each arrow into an “Add delta” node means a weight update is applied; it is not a new full pretrained model. Adapters need compatible model architecture and target layers. Keep the exact base-model identifier and adapter configuration with every saved adapter.

Practical examples

Example A: build and validate one preference record

Problem: Catch missing or empty fields before passing a preference dataset to a trainer.

```
example = {
    "prompt": "Explain what a preference pair contains.",
    "chosen": "It contains a prompt and two candidate answers, one preferred over the other.",
    "rejected": "It only contains one answer.",
}

required = ("prompt", "chosen", "rejected")
missing = [key for key in required if key not in example]
empty = [key for key in required if key in example and not example[key].strip()]

if missing or empty:
    raise ValueError(f"Missing fields: {missing}; empty fields: {empty}")
```

```
print("Valid pair:", example["prompt"])
```

Expected output: `Valid pair: Explain what a preference pair contains.`

Each dictionary key represents the CSV/JSONL column. The `required` tuple defines the trainer contract; the comprehensions collect problems; the final check fails early. In a real dataset, also validate role formatting, duplicate pairs, language, safety, source/consent, and whether chosen is genuinely better for the intended criterion.

Example B: calculate one DPO pair loss

Problem: See the numerical meaning of the DPO margin without loading a model.

```
import math

def softplus_negative(margin):
    # Numerically stable form of log(1 + exp(-margin)).
    return max(0.0, -margin) + math.log1p(math.exp(-abs(margin)))

chosen_log_ratio = 0.20
rejected_log_ratio = -0.10
beta = 0.10
margin = beta * (chosen_log_ratio - rejected_log_ratio)
loss = softplus_negative(margin)

print(f"margin={margin:.2f}, loss={loss:.4f}")
```

Expected output: `margin=0.03, loss=0.6782` (rounding may vary by Python version).

`chosen_log_ratio` and `rejected_log_ratio` are already policy-minus-reference log probabilities. The function computes $\log(1 + e^{-z})$, equivalent to $-\log \sigma(z)$, in a numerically stable way. It demonstrates one pair only; model training averages a batch of losses and computes gradients through the policy log probabilities.

Example C: a tiny LoRA-style update with frozen base weights

Problem: See why a low-rank adapter can be trained while the original matrix stays frozen.

Dependencies: `python -m pip install torch`. This small CPU example is for learning, not language-model training.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(7)
input_size, output_size, rank = 6, 4, 2
base_weight = nn.Parameter(torch.randn(output_size, input_size), requires_grad=False)
adapter_a = nn.Parameter(torch.randn(rank, input_size) * 0.01)
adapter_b = nn.Parameter(torch.zeros(output_size, rank))
alpha = 4.0

inputs = torch.randn(3, input_size)
target = torch.randn(3, output_size)
base_output = F.linear(inputs, base_weight)
low_rank_update = F.linear(F.linear(inputs, adapter_a), adapter_b) * (alpha / rank)
prediction = base_output + low_rank_update
loss = F.mse_loss(prediction, target)
loss.backward()
```

```
print("base gradient:", base_weight.grad)
print("adapter A gradient exists:", adapter_a.grad is not None)
print("adapter B gradient exists:", adapter_b.grad is not None)
```

Expected behavior: the base gradient is `None`; both adapter gradients exist. `adapter_a` maps inputs down to rank 2, `adapter_b` maps them back to output size 4, and `alpha/rank` scales the update. In a transformer these matrices are inserted into selected linear layers; a real LoRA implementation should use PEFT rather than hand-wrapping model layers.

Connecting these examples to the companion notebook

The notebook Preference_Aligned_Training_DPO_ollama.ipynb contains the complete workflow: loading the local CSV, validating columns, loading Qwen with optional 4-bit NF4, attaching LoRA, creating `DPOTrainer`, saving the adapter, merging it, exporting F16 GGUF, and importing it into Ollama. Its dependencies are installed in the notebook's first code cell. Training requires substantial memory and downloads model weights; do not run the full training cell just to try the small examples above.

Comparison: full fine-tuning, LoRA, and QLoRA

Approach	Base weight storage during training	Trainable parameters	Typical advantage	Main cost or limitation
Full fine-tuning	Often FP16/BF16/FP32	Most/all model weights	Maximum flexibility	Highest optimizer and memory requirements; full checkpoint per task
LoRA	Usually unquantized base in a supported dtype	Small low-rank matrices	Smaller trainable state and adapter files	More memory than quantized-base QLoRA; adapters depend on exact base
QLoRA	Frozen base quantized, commonly 4-bit NF4	LoRA matrices	Lower base-weight memory for adaptation	Quantization overhead/quality tradeoffs; hardware and library constraints

The companion notebook chooses QLoRA by default only when CUDA is detected and `USE_4BIT` remains true. Setting `USE_4BIT=False` uses LoRA without 4-bit loading; it still does not make the workflow full fine-tuning. The notebook's later merge reloads an unquantized base, so the deployment artifact is not automatically 4-bit.

What DPO does internally

1. Tokenize the prompt and both candidate responses using the model's expected chat template/format.
2. Compute the policy's log probability for the response tokens of `chosen` and `rejected`.
3. Compute matching log probabilities under a reference policy.
4. Form a relative margin: how much more the current policy favors chosen over rejected, compared with the reference.
5. Apply the logistic loss and backpropagate into trainable parameters. With LoRA, the base is frozen and gradients update adapter parameters.
6. Repeat over batches and epochs; evaluate on held-out prompts and preference judgments.

The DPO objective is not a direct guarantee that every chosen answer becomes absolutely likely. It is a relative comparison under a prompt and reference. A pair can be mislabeled, ambiguous, or encode a narrow preference. DPO can improve the behavior represented by the dataset while weakening unrelated capabilities if data or hyperparameters are poor.

Use cases, trade-offs, and limits

Good fits: response tone/style, concise versus detailed answers, domain-specific answer preferences, ranking helpfulness, and reducing repeatedly observed undesirable response patterns.

Poor assumptions: expecting a tiny dataset to create broad expertise; expecting preference optimization to fact-check claims; assuming that a polite answer is medically safe; assuming one beta, loss, or annotation policy fits every task.

Trade-offs: Pairwise labels communicate relative quality, which can be easier than assigning absolute scores, but collecting comparisons still costs time and introduces annotator disagreement. Reward-model RLHF offers a flexible scalar reward but adds model

training and PPO complexity. DPO is simpler to train from static pairs, but still needs sound data, reference-model handling, and evaluation.

Data quality and annotation guidelines

- Define the goal before labeling: factuality, safety, completeness, tone, or a weighted combination.
- Compare answers to the same prompt; do not let unrelated prompt differences leak into the preference label.
- Write annotation instructions with examples and a tie/uncertain option where supported.
- Use domain experts for specialized or high-impact topics; record provenance and permissions.
- Remove personal or sensitive data unless there is a lawful, reviewed basis to retain it.
- Deduplicate, detect contradictory pairs, check language and formatting, and keep a held-out evaluation set.
- Audit label distributions and annotator disagreement. Disagreement can reveal ambiguous criteria rather than "bad annotators."
- Avoid relying only on synthetic preferences generated by the same model being trained; this can reinforce its errors.

Common mistakes and troubleshooting

Symptom / misconception	Likely explanation	What to check
Model is polite but still wrong	Preference data rewarded style, not verified factuality	Add fact-checked examples and separate factuality evaluation
DPO loss falls but human quality does not improve	Overfitting, noisy preferences, or mismatch between metric and goal	Evaluate held-out prompts; inspect chosen/rejected examples and generations
Chosen/rejected are reversed	Data-label error	Inspect several raw rows before training; correct the data, not the loss sign
CUDA out-of-memory	Batch, context, activations, optimizer state, or model placement exceed VRAM	Reduce sequence length/batch; use gradient accumulation/checkpointing/QLoRA; check other processes
Unsupported LoRA target module	Target names do not exist in that architecture	Inspect model module names and choose supported projection layers
Merge produces a poor or unusable model	Wrong base/adapter pairing, incomplete adapter, dtype or merge issue	Confirm adapter provenance, base revision, tokenizer, merge logs, and a before/after smoke test
Ollama import says unsupported architecture	Importer does not support that architecture/format	Convert the merged Hugging Face model to a supported GGUF using llama.cpp, as in the companion notebook
Windows `UnicodeDecodeError` on command output	Process output was decoded with a locale codec	Capture bytes and explicitly decode UTF-8 with replacement; rerun the edited cell and clear stale notebook output
"LoRA is stacked" causes confusion	A LoRA is a delta, not a complete model	Keep the base and adapter provenance; merge or compose only compatible adapters using documented tooling

Performance, security, and production considerations

- Training and serving are different costs. LoRA reduces trainable state, but activations, sequence length, optimizer memory, and reference-policy computations still matter.
- Record base-model identifier/revision, tokenizer/chat template, dataset version, annotation policy, seeds, package versions, precision, and adapter configuration.
- Keep model and dataset artifacts access-controlled. Review licenses, consent, sensitive data, and supply-chain risks before downloading or deploying models.
- Treat user prompts and generated content as untrusted. Apply input handling, output monitoring, access controls, rate limits, and incident review appropriate to the application.

- For medical, legal, financial, or other high-impact use, require qualified review, domain-specific evaluation, uncertainty handling, and escalation paths. A system prompt or preference loss is not a safety certification.
- Evaluate regressions as well as target improvements: factuality, refusal behavior, harmful content, robustness, latency, throughput, memory, and performance across user groups.
- Quantization is a deployment compression step, not a replacement for training/evaluation. Validate quality after any Q8/Q4 conversion.
- Do not claim "aligned" as a binary property. Alignment is relative to a defined set of goals, users, constraints, and evaluation conditions.

Practice exercises with answers

1. **A row has the same prompt, but `rejected` is actually the better answer. What happens?**
   Answer: The training signal points in the wrong direction for that example. Audit and correct labels; more training can strengthen the error.

2. **The policy and reference assign identical log probabilities to both answers. What is the DPO margin?**
   Answer: Both log-ratio changes are zero, so the margin is zero and the single-pair loss is $-\ln(0.5) = \ln 2 \approx 0.6931$.

3. **Does saving an adapter create an independent model?**
   Answer: No. It stores learned deltas/configuration and needs the compatible base model, unless it is merged into a standalone checkpoint.

4. **Can DPO establish that a selected medical answer is safe?**
   Answer: No. It optimizes the preferences represented by the data. Safety requires expert-reviewed data, explicit evaluation, monitoring, and appropriate clinical boundaries.

5. **The training uses 4-bit QLoRA, but the notebook exports F16 GGUF. Is that inconsistent?**
   Answer: No. Training-time storage, calculation dtype, merge dtype, and deployment quantization are separate stages. This notebook reloads an unquantized base for merge and explicitly exports F16.

Interview questions

1. **How does SFT differ from preference optimization?**
   SFT learns from target demonstrations with token-level likelihood; preference optimization learns from comparative judgments or feedback about candidate outputs.

2. **Why does DPO use a reference policy?**
   The reference anchors the comparison to the starting behavior and appears in the relative log-probability terms; the policy is rewarded for changing preference between chosen and rejected, not merely inflating every response probability.

3. **What is the practical difference between PPO-based RLHF and DPO?**
   The classic PPO pipeline trains a reward model and optimizes sampled outputs with RL. Basic DPO trains directly on preference pairs with a classification-style loss, avoiding a separate reward model/PPO loop.

4. **What is LoRA's low-rank assumption?**
   The useful adaptation can be represented by a lower-rank update BA, allowing a small number of trainable parameters while freezing the original matrix.

5. **Why can preference alignment reduce quality on other tasks?**
   Finite and biased data can shift the policy toward a narrow behavior, cause overfitting, or reward superficial signals that correlate with preference labels but not true quality.

6. **Why is a falling training loss insufficient evidence of success?**
   It only indicates fit to the training objective. Held-out preference tests, quality/safety evaluations, regression checks, and real deployment monitoring are still necessary.

Revision notes

- Pretraining: predict next tokens from text.
- SFT: imitate demonstrations.
- Preference data: prompt + chosen + rejected.
- DPO: raise the chosen-versus-rejected policy/reference margin with a logistic loss.
- RLHF/PPO: commonly train a reward model, then optimize a policy with reinforcement learning.
- LoRA: frozen base plus trainable low-rank deltas; QLoRA quantizes base storage during adapter training.
- Alignment is not truth, safety, or universal correctness.
- Evaluation data must be separate from training data.

Glossary

Term	Definition
Alignment	Shaping model behavior toward stated preferences, goals, and constraints
Adapter	Small parameter set intended to modify a compatible base model
Chosen / rejected	Preferred / less-preferred response in a pair
DPO	Direct Preference Optimization, a direct pairwise preference-training objective
KL divergence	A measure of difference between probability distributions; often used to constrain policy drift
LoRA	Low-Rank Adaptation; trainable low-rank weight updates with base weights frozen
Policy	Model whose response distribution is being optimized
PPO	Proximal Policy Optimization, a policy-gradient reinforcement-learning algorithm
Preference pair	A prompt and two responses with a relative preference label
QLoRA	LoRA training with a quantized, commonly 4-bit, frozen base model
Reference model	Fixed comparison model used in preference objectives or RLHF constraints
Reward model	Model trained to predict a scalar score from human preference labels
RLHF	Reinforcement Learning from Human Feedback; an umbrella family of methods
SFT	Supervised fine-tuning on demonstration input/output examples
Token log probability	Log probability the model assigns to a token given prior context

Recommended next topics

1. Tokenization, causal language modeling, and cross-entropy.
2. Transformer attention and chat templates.
3. Supervised instruction tuning and evaluation splits.
4. Pairwise preference annotation, Bradley-Terry models, and reward modeling.
5. PPO, KL regularization, DPO, IPO, and KTO objectives.
6. LoRA/QLoRA, quantization formats, merging, and adapter provenance.
7. Safety evaluation, red teaming, calibration, and deployment monitoring.

References

- Ouyang et al., Training language models to follow instructions with human feedback, 2022. The InstructGPT paper describes demonstrations, preference rankings, and RLHF stages.
- Rafailov et al., Direct Preference Optimization: Your Language Model is Secretly a Reward Model, 2023, revised 2024. Source for the standard DPO derivation and objective.
- Hu et al., LoRA: Low-Rank Adaptation of Large Language Models, 2021. Source for frozen-base, low-rank adaptation.

- Azar et al., A General Theoretical Paradigm to Understand Learning from Human Preferences, 2023. Discusses preference-optimization objectives including IPO.
- Ethayarajh et al., KTO: Model Alignment as Prospect Theoretic Optimization, 2024. Describes KTO and learning from desirable/undesirable feedback.
- The companion implementation: Preference_Aligned_Training_DPO_ollama.ipynb. APIs and hardware requirements can change; check the installed TRL/Transformers documentation for the environment used to run it.